



Universidad del Desarrollo

Class 3: Introduction to Business Intelligence

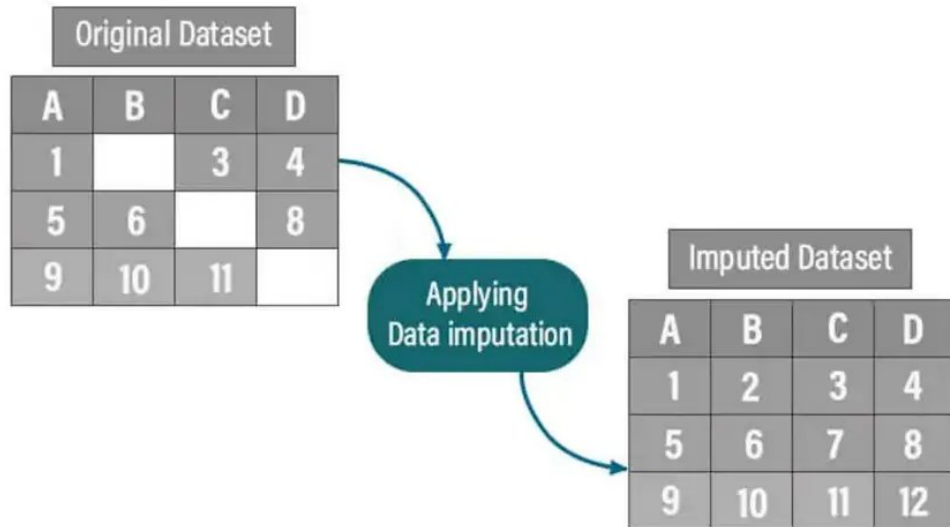
Unit I: Data Cleaning and Transformation

Sebastián Muñoz-Herrera, PhD.
Héctor Hormazábal

Ingeniería


Imputation

Imputation is the process of replacing missing values in a dataset with estimated substitutes, allowing analysis to proceed without discarding incomplete records.



Why this matter?

1

Preserves data volume

Keeps all rows — no information loss from dropping records.

2

Reduces bias

Prevents systematic skew when missingness isn't random.

3

Model compatibility

Most ML algorithms cannot handle NaN values natively.

COMMON STRATEGIES

Statistical

Mean / Median / Mode

Model-based

KNN Imputer

Deletion

dropna

Imputation: KNN Imputer

K-Nearest Neighbors (KNN) Imputation fills in missing values by finding the k most similar rows (neighbors) based on the available features, then using their values to estimate the missing one — typically via mean (for numeric) or mode (for categorical).

Example:

Person	Age	Hours studied	Score
A	20	5	70
B	22	3	60
C	21	4	?
D	25	2	50
E	19	6	80

Imputation: KNN Imputer

Step 1: Calculate the euclidean distance between C to every other point using:

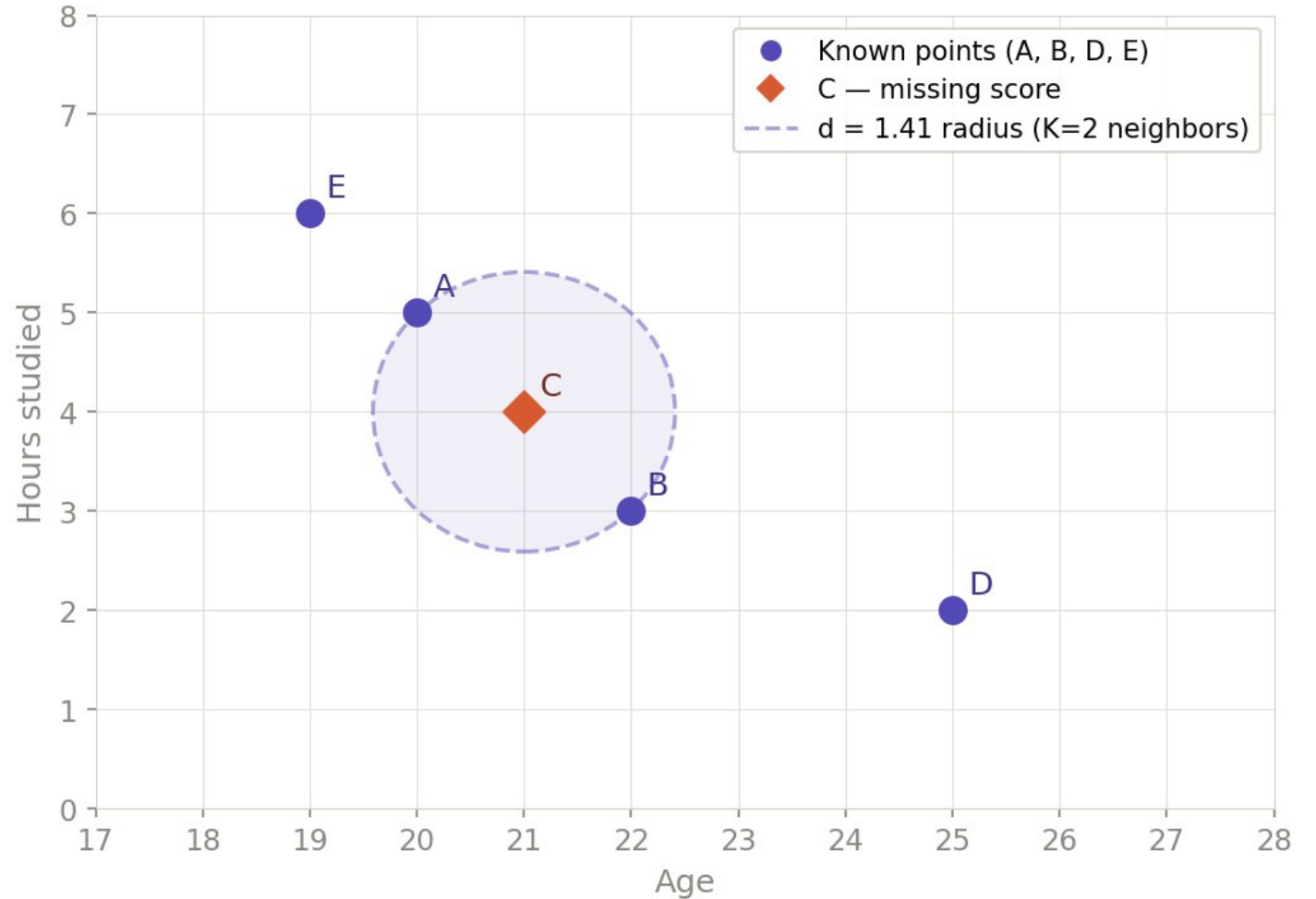
$$d = \sqrt{(Age_i - 21)^2 + (Hours_i - 4)^2}$$

Person	Age diff ²	Hours diff ²	Distance
A	(20-21) ² = 1	(5-4) ² = 1	√(1+1) ≈ 1.41
B	(22-21) ² = 1	(3-4) ² = 1	√(1+1) ≈ 1.41
D	(25-21) ² = 16	(2-4) ² = 4	√(16+4) ≈ 4.47
E	(19-21) ² = 4	(6-4) ² = 4	√(4+4) ≈ 2.83

Imputation: KNN Imputer

Step 2: Pick up the 2-nearest neighbors.

- Sorted by distance:
 - A (1.41)**
 - B (1.41)**
 - E (2.83)
 - D (4.47)
- 2 nearest neighbors are **A** and **B**



Imputation: KNN Imputer

Step 3: Calculate the average score with the selected neighbors.

$$Score_c = \frac{Score_A + Score_B}{2} = 65$$

Person	Age	Hours studied	Score
A	20	5	70
B	22	3	60
C	21	4	65
D	25	2	50
E	19	6	80

Deduplication: Exact and logical duplications

Exact duplications: Rows that are completely identical across all columns — every field matches perfectly.

Fix using : `df.drop_duplicates()`

ID	Name	Age	Score
1	Ana	22	85
1	Ana	22	85

Logical duplicates Rows that represent the same real-world entity but differ in formatting, spelling, or values.

Fix: Needs a business logic. normalize text, match on key fields, use fuzzy matching

D	Name	Age	Score
1	Ana	22	85
1	ANA	22	85
1	Ana García	22	85
2	Ana	22	85

Outlier treatment: Winsorization

A technique to handle outliers by **capping** extreme values at a specified percentile threshold — rather than removing them. The record stays in the dataset; only the extreme value is replaced.

How it works Choose a percentile limit (commonly 5th–95th or 1st–99th). Any value below the lower threshold is replaced with the lower threshold. Any value above the upper threshold is replaced with the upper threshold.

Student	Score	Score - Winsorized (90th percentile)
A	55	55
B	60	60
C	62	62
D	65	65
E	68	68
F	70	70
G	72	72
H	98 ← outlier	72

Outlier treatment: Log-transformation

Log transformation applies the logarithm function to a variable, compressing large values more than small ones. This reduces the effect of extreme outliers and can make a skewed distribution more symmetric.

Why it works The log scale grows slowly — the gap between 10 and 100 becomes the same as the gap between 100 and 1,000. Extreme high values are pulled inward; small values spread out slightly.

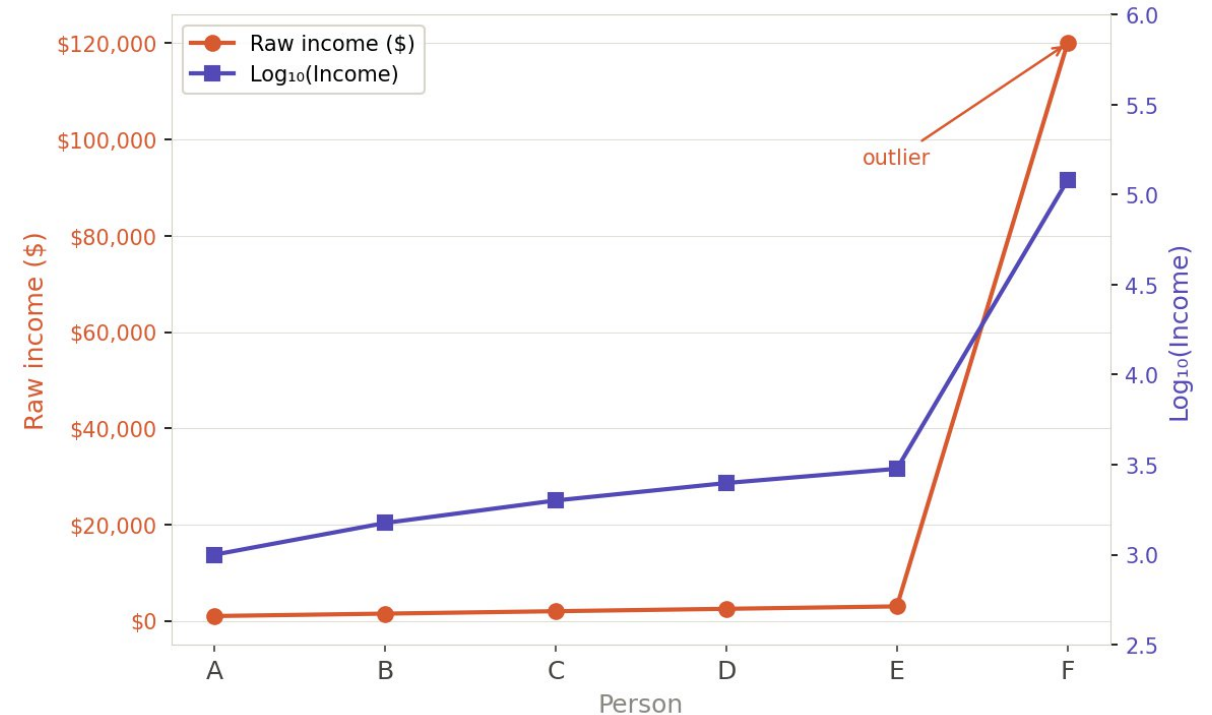
Person	Income (\$)	$\text{Log}_{10}(\text{Income})$
A	1,000	3.00
B	1,500	3.18
C	2,000	3.30
D	2,500	3.40
E	3,000	3.48
F	120,000	5.08

Outlier treatment: Log-transformation

Log transformation applies the logarithm function to a variable, compressing large values more than small ones. This reduces the effect of extreme outliers and can make a skewed distribution more symmetric.

Important Conditions

Condition	Reason
All values must be > 0	$\log(0)$ and $\log(\text{negative})$ are undefined
If zeros exist	use $\log(x + 1)$ — called $\log_{1p}$
Interpretation changes	model results are on the log scale, must back-transform to interpret



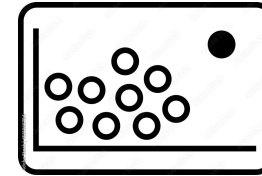
Normalization (Min-Max scaling)

Normalization rescales all values of a variable to a fixed range, typically **0 to 1**, by subtracting the minimum and dividing by the range. It preserves the shape of the original distribution but changes the scale.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$



When to use it When features have different units or scales and the algorithm is sensitive to magnitude — e.g. KNN, neural networks, gradient descent.



Sensitivity to outliers — important caveat If one value is huge in comparison with the others, all other normalized values would be compressed into a tiny range near 0. This is why outliers should be treated *before* normalizing.

Standardization (Standard scaling)

Rescales values so the variable has a mean of 0 and a standard deviation of 1. Unlike normalization, there is no fixed output range — values can be negative or exceed 1.

$$X' = \frac{X - \mu}{\sigma}$$



When to use it Algorithm assumes normality or is sensitive to variance — linear/logistic regression, PCA, SVM.
Data contains outliers — less distorted by extremes than normalization.
Features have different units or scales.



No bounded output — unsuitable for algorithms requiring [0, 1] input
Still sensitive to extreme outliers — $Z > 3$ flags them but doesn't remove them
Fit on training data only — applying to the full dataset causes data leakage

Categorical handling: Label encoding

Converts categorical text values into integers by assigning each unique category a number. Simple and memory-efficient, but introduces an implicit numeric order that may not exist in the data.

Student	Education	Label encoded
A	High school	0
B	Bachelor	1
C	Master	2
D	PhD	3
E	Bachelor	1

Here the encoding is valid — there is a natural order (High school < Bachelor < Master < PhD). Lucky !!

When to use it. The variable is **ordinal** — categories have a meaningful, agreed order. The algorithm is tree-based — decision trees and random forests are not misled by arbitrary integers. Memory or simplicity is a constraint

When NOT to use it. The variable is **nominal** — no natural order exists. The algorithm is distance-based or linear — KNN, SVM, linear regression will be distorted

Categorical handling: One-hot encoding

Converts categorical text values into integers by assigning each unique category a number. Simple and memory-efficient, but introduces an implicit numeric order that may not exist in the data.

Customer	City
A	Paris
B	London
C	Tokyo
D	Paris



Customer	City_Paris	City_London	City_Tokyo
A	1	0	0
B	0	1	0
C	0	0	1
D	1	0	0

When to use it. The variable is **nominal** — no natural order exists. The algorithm is distance-based or linear — KNN, SVM, linear regression, neural networks

When NOT to use it

The variable has **high cardinality** — a city column with 500 unique values produces 500 new columns, causing the curse of dimensionality. Memory is a hard constraint.

Categorical handling: Ordinal encoding

Converts ordered categorical values into integers that **respect the natural order** of the categories. The key difference from label encoding is that the numeric mapping is explicitly defined by the analyst — **not assigned arbitrarily**.

Student	Education	Label encoded
A	High school	0
B	Bachelor	1
C	Master	2
D	PhD	3
E	Bachelor	1

When to use it

The variable is **ordinal** — a clear, meaningful order exists. The order is domain-defined, not inferred automatically. Any algorithm — ordinal encoding is safe for both linear and tree-based models when order is genuine.

Caveats

Equal spacing is assumed — if the real gap between Master and PhD is much larger than Bachelor and Master, the encoding does not capture that. Never apply to nominal variables. Order must be defined manually.



Universidad del Desarrollo

Class 3: Introduction to Business Intelligence

Unit I: Data Cleaning and Transformation

Sebastián Muñoz-Herrera, PhD.
Héctor Hormazábal

Ingeniería
